OpenSSL & Cryptography Infrastructure Cheatsheet

L0 Essence: The essence of cryptographic secure communication is to establish a secure channel over an insecure network using symmetric/asymmetric algorithms ensuring confidentiality, integrity, authenticity, and non-repudiation. Its implementation centers on the TLS handshake state machine, hardware accelerator engines, and secure entropy source maintenance.

[M1.1] Entropy Sources & DRBG Security Boundaries

\textbullet\textbfCore Definition: Cryptographic strength relies on a cryptographically secure Deterministic Random Bit Generator (DRBG) fed by high-entropy sources. \\\textbullet\textbfBottom Mechanism: OpenSSL gathers physical entropy from the OS (such as CPU RdRand instructions and kernel `/dev/urandom` interrupt noise). It feeds this into a NIST SP800-90A compliant DRBG, preventing state backtracking based on computational complexity, ensuring secure key and IV generation.

[M1.2] EVP High-Level Unified Cryptographic API

\textbullet\textbfCore Definition: The EVP API provides a unified, abstract interface for symmetric and asymmetric cryptographic operations. \\\textbullet\textbfBottom Mechanism: EVP decouples application logic from specific algorithm implementations through object-oriented context binding. For instance, passing `EVP_aes_256_gcm()` to `EVP_EncryptInit_ex` allows the code to execute AES-GCM without exposing low-level structural details, enabling seamless Provider swaps.

[M1.3] Symmetric Block Ciphers & Padding Schemes

\textbullet\textbfCore Definition: Block ciphers (e.g., AES) require padding schemes to encrypt data that is not a multiple of the block size. \\\textbullet\textbfBottom Mechanism: Traditional CBC mode chains cipherblocks by XORing the previous ciphertext block with the current plaintext. PKCS

[M1.4] AEAD Authenticated Encryption & Integrity

\textbullet\textbfCore Definition: AEAD ciphers combine encryption and integrity checking into a single operation. \\\textbullet\textbfBottom Mechanism: AES-GCM merges Galois Field multiplication (GMAC) with CTR mode encryption. It outputs the ciphertext alongside a 128-bit authentication Tag. Decryption forces the hardware to verify the Tag against the AAD first. Any bit manipulation triggers authentication failure, preventing ciphertext modification attacks.

[M1.5] Message Digest & HMAC Verification

\textbullet\textbfCore Definition: HMAC is a key-hashed message authentication code that guarantees message integrity and authenticity. \\\textbullet\textbfBottom Mechanism: HMAC uses a nested hashing structure: $HMAC_K(M) = H((K^+ \oplus opad) \parallel H((K^+ \oplus ipad) \parallel M))$. This design mathematically eliminates length extension attacks common in standard hashing functions, preventing signature forge attempts.

[M2.1] Diffie-Hellman & Ephemeral Key Exchange (ECDHE)

\textbullet\textbfCore Definition: Diffie-Hellman allows two parties to agree on a shared secret over an insecure channel. \\\textbullet\textbfBottom Mechanism: Ephemeral Elliptic Curve Diffie-Hellman (ECDHE) generates a private scalar d and public point $Q = d \cdot G$ for each session. The shared secret is derived as $S = d_{client} \cdot Q_{server} = d_{client} d_{server} G$. Discarding the private key after the session ensures Forward Secrecy.

[M2.2] RSA Asymmetric Cipher & OAEP Padding

\textbullet\textbfCore Definition: RSA relies on the mathematical complexity of prime factorization, requiring secure padding for confidentiality. \\\textbullet\textbfBottom Mechanism: Legacy RSA PKCS

[M2.3] ECC Curves & ECDSA Signature Verification

\textbullet\textbfCore Definition: ECC provides asymmetric security based on elliptic curve discrete logarithm problems using shorter keys. \\\textbullet\textbfBottom Mechanism: ECDSA signature generation (r, s) requires a unique random nonce k. If k is reused, the private key can be mathematically extracted. OpenSSL implements RFC 6979 to derive k deterministically from the private key and message hash, eliminating nonce reuse vulnerabilities.

[M2.4] ASN.1 Standards & DER/PEM Serialization

\textbullet\textbfCore Definition: ASN.1 is a standard language for describing structured cryptographic objects such as keys and certificates. \\\textbullet\textbfBottom Mechanism: DER is a binary, space-efficient Distinguished Encoding Rules format utilizing Tag-Length-Value (TLV) encoding. PEM is a Base64 encoded DER file wrapped in headers like `-----BEGIN CERTIFICATE-----` to facilitate transport over text-based networks.

[M2.5] X.509 Certificate Chain Trust Validation

\textbullet\textbfCore Definition: X.509 defines the standard format for public-key certificates linking identities to public keys. \\\textbullet\textbfBottom Mechanism: Validation traverses a tree to a trusted Root CA. The client checks the CA's signature on the certificate: $S = Sign_{CA}(Hash(CertificateBody))$. A valid cryptographic chain validates the server's identity, preventing man-in-the-middle attacks.

[M3.1] TLS 1.2 Handshake & RTT Delay

\textbullet\textbfCore Definition: The TLS 1.2 handshake negotiates security parameters and establishes keys in two network round-trips (2-RTT). \\\textbullet\textbfBottom Mechanism: The steps involve: 1. `ClientHello`/`ServerHello` (cipher suite negotiation); 2. `Certificate` / `ServerKeyExchange` (CA validation and DH parameters); 3. `ClientKeyExchange` (client sends key exchange material); 4. `Finished`. This results in a 2-RTT latency penalty before data transmission.

[M3.2] TLS 1.3 1-RTT Handshake Optimization

\textbullet\textbfCore Definition: TLS 1.3 simplifies the handshake to a single round-trip (1-RTT) to reduce latency. \\\textbullet\textbfBottom Mechanism: The client speculatively includes a Key Share (e.g., X25519 public key) in the `ClientHello`. The server calculates the shared secret immediately, returning its Key Share and encrypted certificate in `ServerHello`. Symmetrical session keys are established in 1-RTT.

L1 Logical Framework

\textbf1.\\textbfNegotiation & Authentication: Client and Server negotiate ciphers and verify identity using X.509 certificate chains within a single round-trip (1-RTT) in TLS 1.3. \\\textbf2.\\textbfAnti-Eavesdropping Key Exchange: Generate ephemeral share keys using ECDHE (Elliptic Curve Diffie-Hellman Ephemeral) to derive shared symmetric session keys providing forward secrecy. \\\textbf3.\\textbfAuthenticated Encryption (AEAD): Employ dual-purpose AEAD ciphers (e.g., AES-GCM) in the record protocol to secure payload data with high-strength symmetric encryption and anti-tamper MAC integrity verification. \\\textbf4.\\textbfHardware Acceleration: Automatically route cryptographic tasks via the EVP abstract layer to CPU AES-NI instructions or dedicated QAT accelerators, eliminating computation bottlenecks.

L2 Data Flow Topology

`ClientHello` $\longrightarrow$ `(Negotiate suites and share keys)` $\longrightarrow$ `ServerHello & Cert` $\longrightarrow$ `(ECDHE secret extract)` $\longrightarrow$ `HKDF key derivation` $\longrightarrow$ `Symmetric key established` $\longrightarrow$ `AEAD Record Encryption` $\longrightarrow$ `Encrypted Application Data`

Trade-off Matrix: Pipeline Overhead & Accuracy

\textbf\textbfDecryption Throughput: Extremely High (Hardware-optimized via CPU AES-NI) $\longrightarrow$ Moderate (Requires two steps: encrypt then authenticate) [Extremely High (Software logic speedup, optimized for CPUs without hardware acceleration)] \\\textbf\textbfHandshake Latency (RTT): 1-RTT (Single round-trip negotiation) $\longrightarrow$ 2-RTT (Traditional TLS 1.2 double round-trip negotiation) [0-RTT (Session resumption data flight)] \\\textbf\textbfSide-channel Exposure: Extremely Low (No table-lookup, algorithm is strictly constant-time) $\longrightarrow$ High (CBC padding leaks timing information) [Extremely Low (Arithmetic-only, resistant to cache side-channels)] \\\textbf\textbfCPU Resource Overhead: Low (Register-level hardware pipeline) $\longrightarrow$ High (Double pass: first AES encrypt, then HMAC digest) [Low (Optimized for mobile platforms without dedicated cryptographic ALUs)]

[M3.3] HKDF Key Derivation Tree

\textbullet\\textbfCore Definition: HKDF is the core key derivation function in TLS 1.3 used to extract and expand keys. \\\textbullet\\textbfBottom Mechanism: Steps include: 1. \textbfExtract: $PRK = HMAC\_Hash(salt, IKM)$ concentrates entropy into a pseudorandom key; 2. \textbfExpand: $OKM = HKDF\_Expand(PRK, info, L)$ generates independent session keys (Client/Server write keys and IVs) iteratively, avoiding cross-key leakage.

[M3.4] Session Tickets & 0-RTT Replay Risks

\textbullet\\textbfCore Definition: Session resumption allows returning clients to skip handshake validation, while 0-RTT enables sending payload in the first packet. \\\textbullet\\textbfBottom Mechanism: The server encrypts session state into a Ticket session bundle given to the client. Upon reconnecting, the client sends this Ticket to restore session state. However, 0-RTT data lack replay protection; attackers can intercept and re-send these packets (Replay Attack). It must be restricted to idempotent GET requests.

[M3.5] TLS Record Protocol Frame Encapsulation

\textbullet\\textbfCore Definition: The Record Protocol is responsible for fragmenting, compressing, and encrypting transport payload. \\\textbullet\\textbfBottom Mechanism: Plaintext is partitioned into frames up to 16KB. Each frame gets a header (type, version, length), and is encrypted using the session AEAD cipher keyed with a monotonic Sequence Number. The receiver checks the sequence number, neutralizing packet injection and out-of-order replay attacks.

[M4.1] OPENSSL_malloc Secure Memory Management

\textbullet\\textbfCore Definition: OpenSSL manages secure memory pools to prevent sensitive key material from leaking to disk or memory space. \\\textbullet\\textbfBottom Mechanism: The allocator locks memory pages (using OS calls like `mlock`), preventing the OS from swapping the pages to swap partitions on disk. It intercepts out-of-memory states and halts immediately on corruption to avoid exploitation.

[M4.2] Timing Attacks & Constant-Time Operations

\textbullet\\textbfCore Definition: Timing attacks extract keys by measuring differences in execution times during cryptographic calculations. \\\textbullet\\textbfBottom Mechanism: Code branch logic like `if (key_bit == 1)` alters CPU execution timing. OpenSSL uses constant-time algorithms for critical calculations, utilizing bitwise masks (`val & mask`) instead of conditional branches to ensure execution time is independent of key values.

[M4.3] Zeroization & Memory Cleansing

\textbullet\\textbfCore Definition: Zeroization ensures that sensitive cryptographic variables are erased immediately after use. \\\textbullet\\textbfBottom Mechanism: Standard `memset` calls can be optimized out by compilers if the memory is not accessed again (Dead-code elimination). OpenSSL implements `OPENSSL_cleanse`, using memory barrier primitives and non-inlined pointers to force zeroing in physical memory.

[M4.4] Heartbleed Vulnerability Root Cause (CVE-2014-0160)

\textbullet\\textbfCore Definition: Heartbleed was a critical vulnerability in OpenSSL's implementation of the TLS Heartbeat extension. \\\textbullet\\textbfBottom Mechanism: The server read a client-reported payload length field without verifying the actual packet size. When allocating the response buffer, it executed `memcpy` based on the fake, larger length, reading adjacent process memory containing session keys and returning it to the attacker.

[M5.1] Side-Channel Cache Timing & RSA Blinding

\textbullet\\textbfCore Definition: Cache timing attacks observe CPU cache misses to infer exponent patterns in modular exponentiation. \\\textbullet\\textbfBottom Mechanism: OpenSSL applies RSA Blinding: it multiplies the plaintext M by a random factor $r^e \pmod N$ before modular exponentiation: $M' = M \cdot r^e \pmod N$. Decryption outputs are multiplied by r^{-1} to remove the bias. This randomizes CPU cache access patterns, nullifying cache analysis.

[M5.2] EVP_PKEY Asymmetric Interface Abstraction

\textbullet\\textbfCore Definition: EVP_PKEY wraps diverse asymmetric algorithms (RSA, ECC, DH) under a unified key abstraction structure. \\\textbullet\\textbfBottom Mechanism: It wraps concrete key structures (`RSA*`, `EC_KEY*`) into a generic `EVP_PKEY` handler. Operations like signature generation (`EVP_PKEY_sign`) use unified wrappers, isolating application logic from changes in the underlying asymmetric curves or math libraries.

[M5.3] SSL_CTX Configuration Context Management

\textbullet\\textbfCore Definition: SSL_CTX is a shared context that configures certificates, trusted CA roots, and protocol parameters for TLS connections. \\\textbullet\\textbfBottom Mechanism: Designed as a singleton wrapper, SSL_CTX loads CA certificates, CRLs, and enforce cipher suites (e.g., disabling SSLv3 and legacy cipher algorithms). Connection-specific `SSL*` structures inherit these settings, optimizing memory.

[M5.4] BIO I/O Stream Pipeline Abstraction

\textbullet\\textbfCore Definition: BIO (Basic Input/Output) is an abstraction layer that wraps files, sockets, memory buffers, and SSL layers into stream pipelines. \\\textbullet\\textbfBottom Mechanism: BIO blocks can be chained together: `BIO_new_ssl` $\rightarrow$ `BIO_push` $\rightarrow$ `BIO_new_socket`. Writing to the top BIO automatically filters data down, encrypting it before writing to the underlying network socket.

[M5.5] OpenSSL Engine Hardware Integration Interface (v1.1.1)

\textbullet\\textbfCore Definition: The Engine interface allowed OpenSSL v1.1.1 to offload cryptographic calculations to external ASIC hardware. \\\textbullet\\textbfBottom Mechanism: It exposes a callback vtable for cryptographic algorithms. Hardware vendors provided dynamic libraries registering implementations (e.g., `ENGINE_set_RSA`), letting OpenSSL dynamically route calls to accelerator cards.

[M6.1] OpenSSL 3.0 Provider Slot Architecture

\textbullet\\textbfCore Definition: Providers replace Engines in OpenSSL 3.0, introducing a modular architecture. \\\textbullet\\textbfBottom Mechanism: Primitives are categorized into namespaces: default, fips, and legacy. Algorithms are queried by name strings (e.g., `"AES-256-GCM"`). This removes compile-time dependencies, letting developers load, unload, and prioritize custom cryptographic providers at runtime.

[M6.2] Intel QAT Cryptographic Core Offloading

\textbullet\\textbfCore Definition: Intel QuickAssist Technology (QAT) offloads heavy cryptographic workloads to external acceleration chips. \\\textbullet\\textbfBottom Mechanism: The QAT Provider routes asymmetric and symmetric operations to physical PCIe accelerators. Tasks are queued to hardware ring buffers asynchronously, allowing the CPU to execute network routing while the chip handles encryption.

[M6.3] AES-NI & CPU Vector Cryptographic Instructions

\textbullet\\textbfCore Definition: Hardware-accelerated instructions built into the CPU (e.g., Intel AES-NI) speed up block encryption. \\\textbullet\\textbfBottom Mechanism: EVP automatically queries CPU capability flags. If supported, it executes assembly code leveraging instructions like `AESENC` / `AESDEC` and `PCLMULQDQ` (carry-less multiplication for GCM). This accelerates GCM throughput by orders of magnitude while reducing CPU load.

[M6.4] Cipher Suite Auditing & Policy Enforcement

\textbullet\\textbfCore Definition: Compliance auditing requires disabling legacy and compromised algorithms in corporate servers. \\\textbullet\\textbfBottom Mechanism: Weak algorithms like RC4 (vulnerable to bias attacks), 3DES (short keys and slow execution), and MD5 (hash collision risks) are disabled. The SSL_CTX blocks these suites via list filters (e.g., `DEFAULT:!RC4:!3DES:!MD5`), preventing downgrade attacks.

Zone T1: EVP/BIO APIs & Keywords

Zone T2: TLS Errors & Protection